

“UPTIME”

1.History,invention and author of the command.

The uptime command has existed since the earliest versions of unix. In early unix systems the uptime was calculated by finding out the difference between the current time and the boot time. The uptime command was first used in the “Berkeley software distribution”(BSD).BSD is a discontinued unix operating system. It was developed and distributed by the Computer Systems Research Group(CSRG) at the University of California,Berkeley. The uptime command was a part of the standard set of commands available on the BSD commands.The specific author of the original “uptime” utility is not widely or easily attributed to a single individual.

2.Uses of the command for a system administrator:

The command provides instant information on the four critical pieces of system administration : current time,system uptime,user count and load averages. The most basic use is to confirm the server is up and running and how long it has been operational since its last boot. The command shows the number of users currently logged in the system.

3.Working of the “uptime” command.

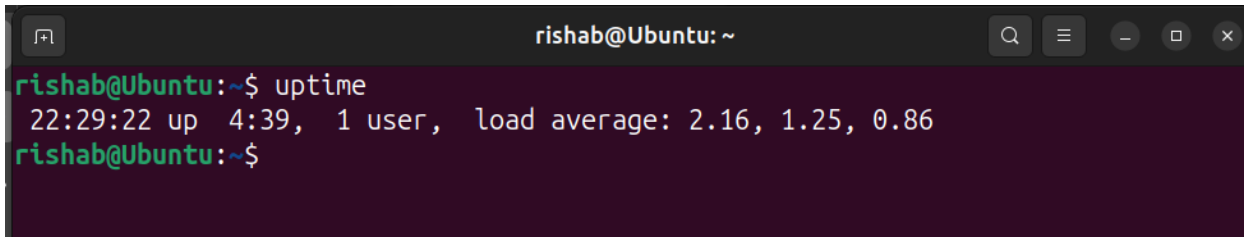
The uptime command works by reading the data directly from the linux kernel and then calculating and formatting that raw information into a human - readable output. The uptime command primarily retrieves its information from two key files within the /proc filesystem,which is a virtual file system maintained by the kernel. The command reads the file /proc/uptime. This file contains two space separated numbers:

Total Uptime:The total number of seconds the system has been running since boot

Total idle time: The total number of seconds the cpu's have been idle.

The command takes the Total Uptime value(the first number),subtracts the current system time from the time of the last boot, and then formats the result into days, hours and minutes.

4.Snapshots

A terminal window titled 'rishab@Ubuntu: ~' with search, menu, and window control icons in the title bar. The terminal shows the command 'uptime' being executed, resulting in the output: '22:29:22 up 4:39, 1 user, load average: 2.16, 1.25, 0.86'. The prompt 'rishab@Ubuntu:~\$' is visible on the line below.

```
rishab@Ubuntu:~$ uptime
22:29:22 up 4:39, 1 user, load average: 2.16, 1.25, 0.86
rishab@Ubuntu:~$
```

Here “22:29:22” suggests the current time at which the command was executed.

Here “up 4:39” suggests the total duration the system has been running since the last boot.

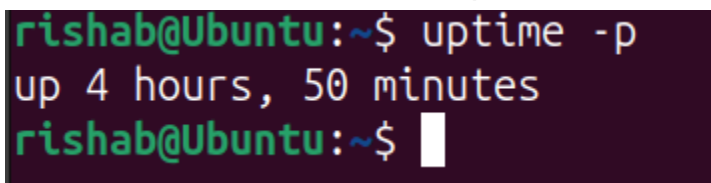
Here “1 user” suggests the count of users currently logged into the system.

Here “load average : 2.16, 1.25, 0.86” suggests the load average over the last 1,5 and 15 minutes.

5. Flags

“Uptime -p”

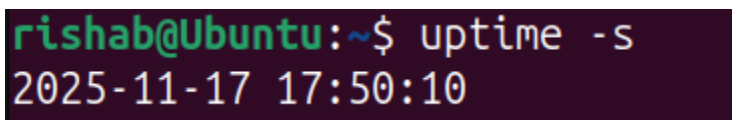
Shows the format in a pretty format. It only shows the duration, not the load averages or current time. It is useful for quick status checks.

A terminal window showing the command 'uptime -p' being executed. The output is 'up 4 hours, 50 minutes'. The prompt 'rishab@Ubuntu:~\$' is visible on the line below.

```
rishab@Ubuntu:~$ uptime -p
up 4 hours, 50 minutes
rishab@Ubuntu:~$
```

“Uptime -s”

Displays the system's date and time since it was booted (the exact time) in YYYY-MM-DD HH:MM:SS format.

A terminal window showing the command 'uptime -s' being executed. The output is '2025-11-17 17:50:10'. The prompt 'rishab@Ubuntu:~\$' is visible on the line below.

```
rishab@Ubuntu:~$ uptime -s
2025-11-17 17:50:10
```

“Uptime -h”

Displays a short help message summarizing all available flags and options. It is useful for quickly checking the commands' full capabilities and syntax.

```
rishab@Ubuntu:~$ uptime -h

Usage:
  uptime [options]

Options:
  -p, --pretty      show uptime in pretty format
  -h, --help        display this help and exit
  -s, --since       system up since
  -V, --version     output version information and exit

For more details see uptime(1).
```

“Uptime -V”

Displays the version information for the “procps-ng” utilities that contain the “uptime” command

```
rishab@Ubuntu:~$ uptime -V
uptime from procps-ng 4.0.4
```

Sources :-

“man uptime”

“<https://www.geeksforgeeks.org/blogs/linux-uptime-command-with-examples/>”

“RKHUNTER”

1.History,invention and author of the command.

In 2003, developer Michael Boelen released the version of Rootkit Hunter. After several years of development, in early 2006, he agreed to hand over development to a development team moving the project to *Sourceforge*. The tool was invented to provide a host-based security check by comparing the SHA-256 (and other) hashes of important system files with a known-good database, looking for files commonly used by rootkits, checking for hidden files and examining suspicious strings in kernel modules.

2.Usages of the command to a system administrator

The “rkhunter” tool is very useful in detecting malicious software that often goes unnoticed by standard antivirus software. It performs checks that are essential for maintaining the integrity, security and stability of a unix like system.

i)its core function is to scan for malicious software designed to hide its presence(rootkits) and programs that grant unauthorized access(backdoors).

ii)if a system binary has been replaced or modified by an attacker to include malicious code, its hash will change. By comparing the current hash with a database of known-good hashes created on a clean system, “rkhunter” can alert the administrator.

iii)it scans for hidden files and directories in suspicious locations. Attackers often use hidden files to stage tools or store logs.

iv)it can look for suspicious listening ports.

v)after running maintenance or applying patches, the admin can re-run the check. Any reported changes highlight files that were legitimately updated or potentially compromised, aiding in change management and auditing.

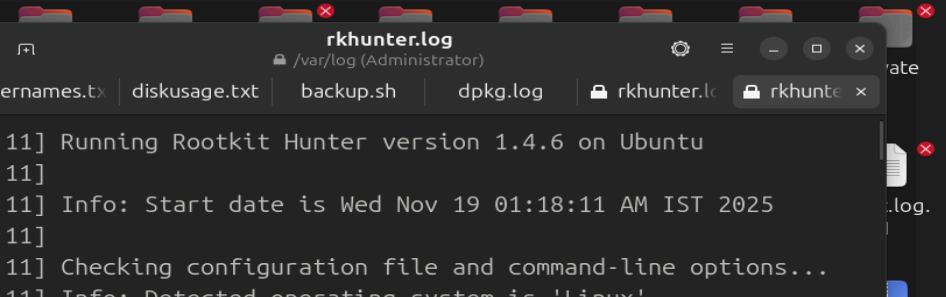
3.The working of the command

i) before scanning it checks the date of its locally stored database of known rootkits and malware. If it is outdated the user should run the “rkhunter –update” command to download the latest signatures of latest malware and rootkits so that we can ensure it can detect the newest threats.

ii) During a scan it recalculates the current hash of each live system file and compares it to the locally stored database. A mismatch signals that the file has been tampered with, potentially by a rootkit or a malware and it signals the user.

iii) Rootkits specialize in hiding their presence by hooking into kernel functions. “Rkhunter” attempts to detect these rootkits by looking for differences between what the system reports and what the scanner discovers. For example, it may look for files in a directory that are not reported by the “ls” command or it may look for open ports that are not reported by the “netstat” command.

iv)After completion “rkhunter” generates a comprehensive report of all findings, which is logged to a specific file (typically in /var/log/rkhunter/). This report includes details of all files scanned, informational messages and clear warnings for any potential rootkits or suspicious anomalies detected.



```
Open ▾  rkhunter.log
/var/log (Administrator)
port.txt | usernames.txt | diskusage.txt | backup.sh | dpkg.log |  rkhunter.log |  rkhunter.log x

[01:18:11] Running Rootkit Hunter version 1.4.6 on Ubuntu
[01:18:11]
[01:18:11] Info: Start date is Wed Nov 19 01:18:11 AM IST 2025
[01:18:11]
[01:18:11] Checking configuration file and command-line options...
[01:18:11] Info: Detected operating system is 'Linux'
[01:18:11] Info: Found O/S name: Ubuntu 24.04.3 LTS
[01:18:11] Info: Command line is /usr/bin/rkhunter --check
[01:18:11] Info: Environment shell is /bin/bash; rkhunter is using
dash
[01:18:11] Info: Using configuration file '/etc/rkhunter.conf'
[01:18:11] Info: Installation directory is '/usr'
[01:18:11] Info: Using language 'en'
[01:18:11] Info: Using '/var/lib/rkhunter/db' as the database
directory
[01:18:11] Info: Using '/usr/share/rkhunter/scripts' as the support
script directory
```

4.Snapshots of the execution

```
rishab@Ubuntu:~$ sudo rkhunter --check  
[sudo] password for rishab:  
[ Rootkit Hunter version 1.4.6 ]
```

Checking system commands...

Performing 'strings' command checks

Checking 'strings' command [OK]

Performing 'shared libraries' checks

Checking for preloading variables [None found]

Checking for preloaded libraries [None found]

Checking LD_LIBRARY_PATH variable [Not found]

Performing file properties checks

Checking for prerequisites [OK]

/usr/sbin/adduser [OK]

/usr/sbin/chroot [OK]

/usr/sbin/cron [OK]

/usr/sbin/depmod [OK]

/usr/sbin/fsck [OK]

/usr/sbin/groupadd [OK]

/usr/sbin/groupdel [OK]

/usr/sbin/groupmod [OK]

/usr/sbin/grpck [OK]

/usr/sbin/ifconfig [OK]

/usr/sbin/init [OK]

/usr/sbin/insmod [OK]

/usr/sbin/ip [OK]

/usr/sbin/lsmmod [OK]

/usr/sbin/modinfo [OK]

/usr/sbin/modprobe [OK]

/usr/sbin/nologin [OK]

/usr/sbin/pwck [OK]

/usr/sbin/rmmod [OK]

/usr/sbin/route [OK]

```

/usr/bin/systemd [ OK ]
/usr/bin/systemctl [ OK ]
/usr/bin/mawk [ OK ]
/usr/bin/lwp-request [ Warning ]
/usr/bin/plocate [ OK ]
/usr/bin/bsd-mailx [ OK ]
/usr/bin/dash [ OK ]
/usr/bin/x86_64-linux-gnu-size [ OK ]
/usr/bin/x86_64-linux-gnu-strings [ OK ]
/usr/bin/inetutils-telnet [ OK ]
/usr/bin/which.debianutils [ OK ]
/usr/lib/systemd/systemd [ OK ]

```

There is a warning for the “/usr/bin/lwp-request”.

```

rishab@Ubuntu:~$ cat /usr/bin/lwp-request
#!/usr/bin/perl

# Simple user agent using LWP library.

=head1 NAME

lwp-request - Simple command line user agent

=head1 SYNOPSIS

B<lwp-request> [B<-afPuUsSedvhx>] [B<-m> I<method>] [B<-b> I<base URL>] [B<-t> I<timeout>]
               [B<-i> I<if-modified-since>] [B<-c> I<content-type>]
               [B<-C> I<credentials>] [B<-p> I<proxy-url>] [B<-o> I<format>] I<url>...

=head1 DESCRIPTION

This program can be used to send requests to WWW servers and your
local file system. The request content for POST, PUT and PATCH
methods is read from stdin. The content of the response is printed on
stdout. Error messages are printed on stderr. The program returns a
status value indicating the number of URLs that failed.

The options are:

=over 4

=item -m <method>

Set which method to use for the request. If this option is not used,
then the method is derived from the name of the program.

=item -f

```

Checking for rootkits...

Performing check of known rootkit files and directories

55808 Trojan - Variant A	[Not found]
ADM Worm	[Not found]
AjaKit Rootkit	[Not found]
Adore Rootkit	[Not found]
aPa Kit	[Not found]
Apache Worm	[Not found]
Ambient (ark) Rootkit	[Not found]
Balaur Rootkit	[Not found]
BeastKit Rootkit	[Not found]
beX2 Rootkit	[Not found]
BOBKit Rootkit	[Not found]
cb Rootkit	[Not found]
CiNIK Worm (Slapper.B variant)	[Not found]
Danny-Boy's Abuse Kit	[Not found]
Devil RootKit	[Not found]
Diamorphine LKM	[Not found]
Dica-Kit Rootkit	[Not found]
Dreams Rootkit	[Not found]
Duarawkz Rootkit	[Not found]
Ebury backdoor	[Not found]
Enye LKM	[Not found]
Flea Linux Rootkit	[Not found]
Fu Rootkit	[Not found]
Fuck`it Rootkit	[Not found]
GasKit Rootkit	[Not found]
Heroin LKM	[Not found]
HjC Kit	[Not found]
ignoKit Rootkit	[Not found]
IntoXonia-NG Rootkit	[Not found]
Irix Rootkit	[Not found]
Jynx Rootkit	[Not found]
Jynx2 Rootkit	[Not found]

Here the “rkhunter” tool is performing checks on known rootkits and directories.

```
Performing additional rootkit checks
  Suckit Rootkit additional checks          [ OK ]
  Checking for possible rootkit files and directories [ None found ]
  Checking for possible rootkit strings       [ None found ]

Performing malware checks
  Checking running processes for suspicious files [ None found ]
  Checking for login backdoors                  [ None found ]
  Checking for sniffer log files                [ None found ]
  Checking for suspicious directories           [ None found ]
  Checking for suspicious (large) shared memory segments [ None found ]
  Checking for Apache backdoor                 [ Not found ]

Performing Linux specific checks
  Checking loaded kernel modules               [ OK ]
  Checking kernel module names                 [ OK ]
```


5.Flags

“rkhunter --check”

This command tells the “rkhunter” to perform various security checks on the local system. This is the command used to check if the system is currently compromised.

```
rishab@Ubuntu:~$ sudo rkhunter --check
[sudo] password for rishab:
[ Rootkit Hunter version 1.4.6 ]

Checking system commands...

Performing 'strings' command checks
  Checking 'strings' command [ OK ]

Performing 'shared libraries' checks
  Checking for preloading variables [ None found ]
  Checking for preloaded libraries [ None found ]
  Checking LD_LIBRARY_PATH variable [ Not found ]

Performing file properties checks
  Checking for prerequisites [ OK ]
  /usr/sbin/adduser [ OK ]
  /usr/sbin/chroot [ OK ]
  /usr/sbin/cron [ OK ]
  /usr/sbin/depmod [ OK ]
  /usr/sbin/fsck [ OK ]
  /usr/sbin/groupadd [ OK ]
  /usr/sbin/groupdel [ OK ]
  /usr/sbin/groupmod [ OK ]
  /usr/sbin/grpck [ OK ]
  /usr/sbin/ifconfig [ OK ]
  /usr/sbin/init [ OK ]
  /usr/sbin/inssmod [ OK ]
  /usr/sbin/ip [ OK ]
  /usr/sbin/lsmmod [ OK ]
  /usr/sbin/modinfo [ OK ]
  /usr/sbin/modprobe [ OK ]
  /usr/sbin/nologin [ OK ]
  /usr/sbin/pwck [ OK ]
  /usr/sbin/rmmod [ OK ]
  /usr/sbin/route [ OK ]
```

"rkhunter --update"

This is the command used to update the rkhunter tool or to download the latest signature files and data from the rootkit hunter project. This keeps the tool current with new rootkit and malware definitions.

```
rishab@Ubuntu:~$ sudo rkhunter --update
[ Rootkit Hunter version 1.4.6 ]

Checking rkhunter data files...
  Checking file mirrors.dat           [ Updated ]
  Checking file programs_bad.dat      [ No update ]
  Checking file backdoorports.dat     [ No update ]
  Checking file suspscan.dat          [ No update ]
  Checking file i18n/cn               [ Skipped ]
  Checking file i18n/de               [ Skipped ]
  Checking file i18n/en               [ No update ]
  Checking file i18n/tr               [ Skipped ]
  Checking file i18n/tr.utf8          [ Skipped ]
  Checking file i18n/zh               [ Skipped ]
  Checking file i18n/zh.utf8          [ Skipped ]
  Checking file i18n/ja               [ Skipped ]
```

"rkhunter --versioncheck"

This command option causes rkhunter to check if there is a later version of the program. A command-line web browser must be present on the system when using this option.

```
rishab@Ubuntu:~$ sudo rkhunter --versioncheck
[ Rootkit Hunter version 1.4.6 ]

Checking rkhunter version...
  This version   : 1.4.6
  Latest version: 1.4.6
```

"rkhunter --propupd"

One of the checks rkhunter performs is to compare various current file properties of various commands, against those it has previously stored. This command option causes rkhunter to update its data file of stored values with the current values.

```
rishab@Ubuntu:~$ sudo rkhunter --propupd
[sudo] password for rishab:
[ Rootkit Hunter version 1.4.6 ]
File updated: searched for 182 files, found 146
```

"rkhunter --list"

This command option will list some of the supported capabilities of the program, and then exit. The tests option lists the currently available test names.

```
Perl module installation status:
perl command           Installed
File::stat             Installed
Getopt::Long           Installed
Crypt::RIPEMD160       MISSING
Digest::MD5            Installed
Digest::SHA            Installed
Digest::SHA1           MISSING
Digest::SHA256         MISSING
Digest::SHA::PurePerl  MISSING
Digest::Whirlpool      MISSING
LWP                    Installed
URI                    Installed
HTTP::Status           Installed
HTTP::Date             Installed
Socket                Installed
Carp                   Installed
```

“rkhunter --help”

The rkhunter --help command displays a summary of the most common and essential command -line options(flags) available for the rootkit hunter tool.

```
rishab@Ubuntu:~$ rkhunter --help

Usage: rkhunter [--check | --unlock | --update | --versioncheck |
               --propupd [{filename | directory | package name},...] |
               --list [{tests | {lang | languages} | rootkits | perl | propfiles}] |
               --config-check | --version | --help] [options]

Current options are:
    --append-log           Append to the logfile, do not overwrite
    --bindir <directory>... Use the specified command directories
    -c, --check            Check the local system
    -C, --config-check     Check the configuration file(s), then exit
    --cs2, --color-set2    Use the second color set for output
    --configfile <file>    Use the specified configuration file
    --cronjob              Run as a cron job
                           (implies -c, --sk and --nocolors options)
    --dbdir <directory>    Use the specified database directory
    --debug                Debug mode
                           (Do not use unless asked to do so)
    --disable <test>[,<test>...] Disable specific tests
                           (Default is to disable no tests)
    --display-logfile      Display the logfile at the end
```

Source : man command
